

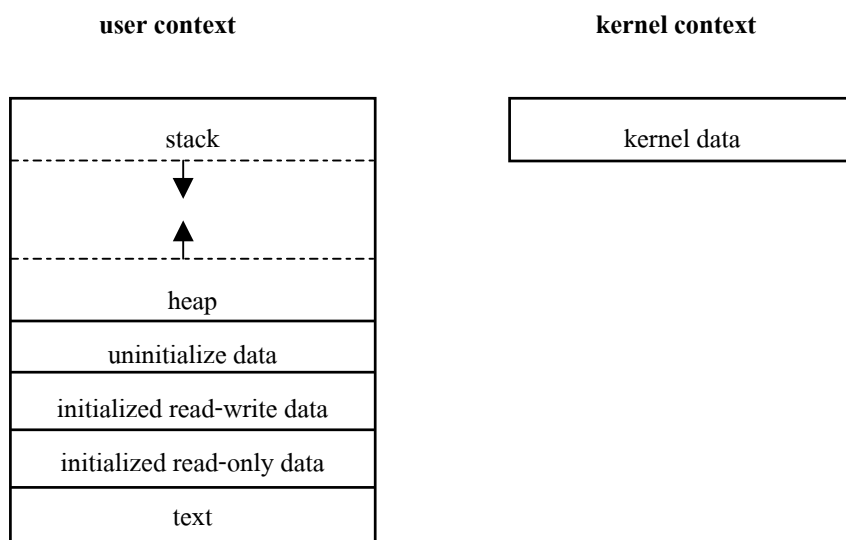
บทที่ 2

โปรเซสและข้อมูลเกี่ยวกับโปรเซส

2.1. ความหมายของโปรเซส

โปรเซส หมายถึง ชุดคำสั่งในหน่วยความจำที่กำลังทำงานอยู่ในขณะนั้น โดยมีชุดควบคุมการทำงาน 1 ชุดเป็นของตัวเองซึ่งได้ใช้ทรัพยากรของระบบไปส่วนหนึ่ง ทั้งนี้ต้องทำความเข้าใจไว้ก่อนว่า โปรเซสกับโปรแกรมไม่ใช่สิ่งเดียวกันยกตัวอย่างเช่น ในระบบมัลติยูสเซอร์สมมติว่ามีผู้ใช้งาน 50 คน กำลังใช้โปรแกรม pine เพื่อทำการอ่านอีเมลพร้อมๆ กันก็จะมีโปรเซสเกิดขึ้น 50 โปรเซสที่กำลังใช้โปรแกรม pine เพียงโปรแกรมเดียว

2.2. โครงสร้างของโปรเซส



รูปที่ 2.1 แสดงโครงสร้างอย่างคร่าว ๆ ของโปรเซส

จากรูปเราแบ่งโครงสร้างโปรเซส 2 ส่วน คือ

ส่วน user context

ประกอบไปด้วยส่วนต่าง ๆ ดังนี้

- **text** เก็บภาษาเครื่อง (machine instruction) ที่ใช้รัน โดยปกติถูกกำหนดให้มีเพอร์มิสชันเป็น read only ดังนั้นโปรเซสจึงไม่สามารถแก้ไขข้อมูลส่วนนี้ได้
- **data** เก็บข้อมูลที่ใช้ในโปรแกรม แบ่งเป็น 3 ส่วนย่อยคือ uninitialized data, initialized read-write data และ initialized read only data

- **heap** ใช้งานในขณะรันไทม์ เพื่อกำหนดพื้นที่หน่วยความจำให้เพียงพอต่อความต้องการในการใช้งาน
- **stack** สำหรับควบคุมการใช้งานตัวแปร การเรียกใช้ฟังก์ชันและการส่งค่ากลับ

ส่วน kernel context

ถูกควบคุมและใช้งานโดยเคอร์เนลเท่านั้น เพื่อเก็บข้อมูลที่จำเป็นสำหรับควบคุมและติดตามการทำงานของโปรเซสซึ่งการที่เคอร์เนลจะจัดการกับโปรเซสได้นั้น ตัวเคอร์เนลเองต้องทราบข้อมูลต่างๆ ที่เกี่ยวกับโปรเซสแต่ละตัวเก็บไว้เช่น มีหมายเลขประจำโปรเซส (process ID) มีค่าเท่าไร สถานะการทำงานของโปรเซสกำลังประมวลผลอยู่หรือถูกบล็อก (block) อยู่หรือไม่ มีแอดเดรสสเปซ (address space) อยู่ที่ตำแหน่งใด มีไพโอริตีเท่าไร เป็นต้น ซึ่งข้อมูลต่าง ๆ เหล่านี้ แต่ละโปรเซสมีเก็บไว้ในแต่ละโปรเซสในส่วนที่เรียกว่า process descriptor โดยรายละเอียดโครงสร้างที่เป็นส่วนประกาศในภาษาซี สามารถดูได้ที่ `/usr/include/linux/sched.h` ซึ่งอาจจะแตกต่างกันไปตามเวอร์ชันของเคอร์เนล

```

struct task_struct {
    /*
     * offsets of these are hardcoded elsewhere - touch with care
     */
    volatile long state; /* -1 unrunnable, 0 runnable, >0 stopped */
    unsigned long flags; /* per process flags, defined below */
    int sigpending;
    mm_segment_t addr_limit; /* thread address space:
                               0-0xBFFFFFFF for user-thread
                               0-0xFFFFFFFF for kernel-thread
                               */
    struct exec_domain *exec_domain;
    volatile long need_resched;
    unsigned long ptrace;

    int lock_depth; /* Lock depth */

    /*
     * offset 32 begins here on 32-bit platforms. We keep
     * all fields in a single cacheline that are needed for
     */
};

```

รูปที่ 2.2 โครงสร้างของ process descriptor ในไฟล์ `/usr/include/linux/sched.h`

ในโครงสร้างข้อมูลนี้ มีโครงสร้างข้อมูลที่ใช้ชี้ไปยัง process descriptor ถัดไปและก่อนหน้า ซึ่งหากลองพิจารณาจะเห็นว่าโครงสร้างเป็น circular doubly linked list ดังรูปที่ 2.3 ซึ่งส่วนหัวของลิงค์ลิสต์นี้เป็น `init_task` descriptor ซึ่งเป็นโปรเซสแรกของลินุกซ์ที่เกิดขึ้นในช่วงการบูตระบบซึ่งมักถูกเรียกว่า process 0 หรือ swapper จะมีหน้าที่ในการบริหารงานโปรเซส

คำสั่ง top

แสดงข้อมูลของโปรเซสเช่นกัน แต่มีการสแกนข้อมูลอยู่ตลอดเวลา

```

1:21am up 1:59, 3 users, load average: 0.29, 0.29, 0.27
63 processes: 58 sleeping, 5 running, 0 zombie, 0 stopped
CPU states: 5.6% user, 18.4% system, 0.0% nice, 75.8% idle
Mem: 191312K av, 18740K used, 3912K free, 0K shrd, 6464K buff
Swap: 136544K av, 0K used, 136544K free

  PID USER   PRI  NI  SIZE  RSS SHARE STAT  %CPU  %MEM  TIME COMMAND
 1673 mint    16   0 45884  44M 23824 R   25.3  23.9  5:02 mozilla-bin
   851 root     13   0 103M 103M 13316 R   15.0  55.5  1:31 X
 1964 root     14   0 1056 1056   840 R    5.6   0.5  0:00 top
   955 mint     9   0 10928  10M  8796 S    1.8   5.7  0:05 kdeinit
   951 mint     9   0  8536  8536  7012 S    0.9   4.4  0:03 kdeinit
     1 root      8   0   528   528   460 S    0.0   0.2  0:05 init
     2 root      9   0     0     0     0 SW    0.0   0.0  0:00 keventd
     3 root      9   0     0     0     0 SW    0.0   0.0  20:46 kapp-idled
     4 root      9   0     0     0     0 SW    0.0   0.0  0:00 kswapd
     5 root      9   0     0     0     0 SW    0.0   0.0  0:00 kreclamd
     6 root      9   0     0     0     0 SW    0.0   0.0  0:00 bdflush
     7 root      9   0     0     0     0 SW    0.0   0.0  0:00 kupdated
     8 root     -1 -20   0     0     0 SW<   0.0   0.0  0:00 mdrecoveryd
    329 root      9   0     0     0     0 SW    0.0   0.0  0:00 khubd
  
```

รูปที่ 2.5 แสดงผลลัพธ์ของการรันคำสั่ง top

2.4. การจัดเก็บข้อมูลใน /proc

ภายในไดเรกทอรี /proc เป็นที่เก็บข้อมูลรายละเอียดเกี่ยวกับเคอร์เนลและโปรเซสซึ่งในที่นี้ขอก้าวถึงเฉพาะในส่วนที่เกี่ยวกับโปรเซส

/proc/[number]

เก็บข้อมูลเกี่ยวกับโปรเซสตามหมายเลขที่กำกับไว้ เช่น /proc/1243 ภายในไดเรกทอรีนี้เก็บข้อมูลเกี่ยวกับโปรเซสที่มีหมายเลขโปรเซสเป็น 1243 ไว้ ซึ่งภายในมีรายละเอียดดังนี้

1. ไฟล์ cmdline

เก็บคำสั่งที่โปรเซสนั้นประมวลผล

2. ไฟล์ environ

เก็บค่าตัวแปรสภาพแวดล้อมของโปรเซสนั้น ๆ

3. ไดเรกทอรี fd

เก็บ symbolic link ของ file descriptor ของไฟล์ที่โปรเซสนั้นได้เรียกใช้งาน

4. cwd Symbolic link

เป็นลิงก์ที่ชี้ไปยังไดเรกทอรีที่โปรเซสทำงานอยู่ (current working directory)

5. exe Symbolic link

เป็นลิงก์ที่ชี้ไปยังไฟล์ที่โปรเซสกำลังประมวลผลอยู่

6. ไฟล์ *maps*

เก็บส่วน memory map ของโปรเซส ซึ่งประกอบด้วยช่วงตำแหน่ง , permission, offset เป็นต้น

7. root Symbolic link

เป็นลิงก์ไปยังรูตไดเรกทอรี (root directory)

8. ไฟล์ *statm*

เก็บข้อมูลการใช้งานหน่วยความจำของโปรเซส

9. ไฟล์ *stat*

เก็บข้อมูลรายละเอียดเกี่ยวกับโปรเซส ซึ่งไฟล์นี้เป็นไฟล์ที่มีรายละเอียดมากที่สุดโดยมีการเก็บข้อมูลเรียง ๆ กันซึ่งลำดับการเรียงข้อมูลนั้นอาจแตกต่างกันไปตามเวอร์ชันของเคอร์เนล (ซึ่งในระบบที่ใช้เป็นเป็นเคอร์เนล 2.4.3 ซึ่งสามารถตรวจสอบลำดับของข้อมูลได้ใน source code ของเคอร์เนลที่ /usr/src/linux/fs/proc/array.c) ซึ่งความหมายของแต่ละตัวแปรดูได้ตามตารางที่ 2.1

ตัวแปร	ความหมาย
pid	หมายเลขประจำโปรเซส
cmd	คำสั่งที่โปรเซสทำการประมวลผล
state	สถานะของโปรเซส
ppid	หมายเลขประจำโปรเซสของโปรเซสที่เป็นผู้สร้างโปรเซสนี้
pgrp	หมายเลขกลุ่มของโปรเซส
session	หมายเลข session ของโปรเซส
tty	tty ที่โปรเซสใช้
tpgid	หมายเลขประจำโปรเซสของ tty ที่ใช้
flags	flag ของโปรเซส
minflt	ค่า minor page fault
cminflt	ค่า minor page fault ที่โดยรวมค่าที่โปรเซสลูกของโปรเซสนี้ด้วย
majflt	ค่า major page fault
cmajflt	ค่า major page fault ที่โดยรวมค่าที่โปรเซสลูกของโปรเซสนี้ด้วย
utime	User time ในหน่วย jiffies (1/100 วินาที)
stime	System time ในหน่วย jiffies
cutime	User time โดยรวมเวลาของโปรเซสลูกด้วย ในหน่วย jiffies

ตารางที่ 2.1 แสดงความหมายของข้อมูลในไฟล์ /proc/[number]/stat

ตัวแปร	ความหมาย
cstime	System time โดยรวมเวลาของโปรเซสลูกด้วย ในหน่วย jiffies
priority	ค่า static priority ของโปรเซส
nice	เป็นค่าไพริอริตี้ที่สามารถเปลี่ยนแปลงได้ (Dynamic priority)
start_time	เวลาที่โปรเซสเริ่มทำงาน
vsize	ขนาดของ Visual Memory ในหน่วย byte
rss	Resident set size
rlim	ค่าจำกัดของ RSS
start_code	จุดเริ่มต้นของโค้ดเซ็กเมนต์ (code segment)
end_cod	จุดสิ้นสุดของโค้ดเซ็กเมนต์
start_stack	จุดเริ่มต้นของสแต็กเซ็กเมนต์ (stack segment)
esp	ตำแหน่งของสแต็กเฟรมปัจจุบัน (current stack frame)
eip	ตำแหน่งของสแต็กเฟรมปัจจุบัน (current stack frame)
pending.signal	เก็บสัญญาณที่โปรเซสต้องการตรวจสอบว่ายังค้างอยู่ในระบบหรือไม่
blocked.signal	เก็บสัญญาณที่โปรเซสต้องการบล็อก
Sigign	เก็บสัญญาณที่โปรเซสไม่สนใจ
Sigcatch	เก็บสัญญาณที่โปรเซสกำลังรอ

ตารางที่ 2.1 แสดงความหมายของข้อมูลในไฟล์ /proc/[number]/stat (ต่อ)

10. ไฟล์ status

เก็บข้อมูลของโปรเซสเช่นเดียวกันโดยแสดงให้เห็นอยู่ในรูปที่เข้าใจได้ง่าย แต่ข้อมูลน้อยกว่าในไฟล์ stat

2.5. ล็อกไฟล์ acct

ไฟล์นี้มีหน้าที่เก็บข้อมูลของโปรเซสที่ผู้ใช้ได้เรียกใช้งานซึ่งถูกบันทึกเมื่อโปรเซสจบการทำงาน ลักษณะโครงสร้างจะสามารถเข้าไปดูได้ที่ /usr/include/linux/acct.h ซึ่งมีลักษณะดังนี้

ตัวแปร	คำอธิบาย
ac_flag	Flags ของโปรเซส
ac_uid	หมายเลขประจำตัวของผู้ที่เป็นเจ้าของโปรเซส
ac_gid	หมายเลขประจำกลุ่มของผู้ที่เจ้าของโปรเซส

ตัวแปร	คำอธิบาย
ac_tty	เทอร์มินอลที่ใช้ในการรันโปรแกรม
ac_btime	เวลาที่โปรแกรมเกิดขึ้น
ac_untime	User time
ac_stime	System Time
ac_etime	Elapsed Time
ac_mem	ปริมาณการใช้หน่วยความจำโดยเฉลี่ย
ac_io	ปริมาณการใช้ i/o
ac_rw	จำนวนบล็อกที่เขียนหรืออ่าน
ac_minflt	ค่า Minor Pagefaults
ac_majflt	ค่า Major Pagefaults
ac_swaps	จำนวน swap ที่ใช้
ac_exitcode	Exit code ของโปรแกรม
ac_comm[ACCT_COMM + 1]	คำสั่งที่ใช้ในโปรเซสนี้

ตารางที่ 2.2 แสดงความหมายของตัวแปรในโครงสร้าง struct acct

คำสั่งที่ใช้ตรวจสอบดูข้อมูลในไฟล์นี้ก็มีเช่น sa เป็นต้น

```

[root@music01 oyoyo]# sa | more
137697 322911,64re 338,00cp 0avio 758k
11 107,89re 96,35cp 0avio 4794k vmware
124 2680,13re 81,27cp 0avio 4204k mozilla-bin
70 8620,97re 46,41cp 0avio 3009k X
1992 76082,56re 21,40cp 0avio 4188k kdeinit*
50 5382,03re 11,46cp 0avio 1155k artsd
506 10,80re 7,19cp 0avio 1593k ccl
10485 10,85re 4,78cp 0avio 362k w
16423 9,00re 4,75cp 0avio 538k bzcatt
452 6,04re 4,09cp 0avio 572k ttmkfdir
8 228,14re 2,48cp 0avio 504k top
123 1181,83re 2,36cp 0avio 5953k xams*
197 7787,66re 2,19cp 0avio 2877k kdm*
101 3,08re 1,98cp 0avio 3906k kdm.desktop
52 8651,54re 1,74cp 0avio 1249k xfs*
16 37,39re 1,56cp 0avio 3805k grpmi*
49 25,17re 1,42cp 0avio 4201k ksysguard
--More--

```

รูปที่ 2.6 แสดงสถิติของผู้ใช้งานโดยคำสั่ง sa